

KFKA – Guia das páginas HTML para CSS e JS

Sep 22, 2026 · @Someone

Visão geral

O HTML das 15 páginas está pronto; falta o CSS (visual e responsividade) e o JS (dados, validação e navegação). Cada perfil tem uma página inicial e mais 3 páginas, todas com a mesma estrutura: cabeçalho com menu, trilha de navegação, seções com <h2> e rodapé.

```
public/
├── login.html
├── 404.html
├── css/          ← CSS (a criar)
├── js/           ← JS (a criar)
├── responsavel/ inicio · relatorios · relatorio · conta
├── professor/   inicio · turmas · acompanhamento · conta
└── admin/       inicio · revisao · revisar · conta
```

O fluxo principal do sistema passa pelos três perfis:

```
flowchart LR
    A[Professor<br/>acompanhamento.html] -->|Enviar para revisão| B[Admin<br/>revisar.html]
    B -->|Devolver| A
    B -->|Publicar| C[Responsável<br/>relatorio.html]
    C -->|Registrar ciência| D[Professor vê o retorno<br/>acompanhamento.html]
```

Nenhuma página traz dados escritos no HTML: tudo começa com "Carregando..." ou "-" e o JS substitui pelo que vier da API.

Convenções de todas as páginas

Estas regras valem para as 15 páginas; resolva uma vez e reaproveite.

Elemento	O que é	O que fazer
#nome-usuario	Nome no cabeçalho	JS coloca o nome do usuário logado
#botao-sair	Botão "Sair"	JS encerra a sessão e volta para /login.html
[aria-current="page"]	Item do menu da página atual	CSS destaca esse item (use o atributo como seletor)
<nav aria-label="Você está em">	Trilha de navegação	CSS: itens na mesma linha, separados por ›
<tbody id="tabela-..."> com "Carregando..."	Tabela vazia esperando dados	JS troca pelas linhas; se vier vazio, "Nenhum resultado"; se a API falhar, a mensagem de erro

Elemento	O que é	O que fazer
<code></code> ou <code><dd></code> com "Carregando..." ou "-"	Lista ou valor esperando dados	Mesmo tratamento das tabelas
<code>role="alert" + hidden</code>	Mensagem geral de um formulário	JS escreve o texto e tira o hidden
<code></code>	Erro de um campo	JS escreve o erro e põe <code>aria-invalid="true"</code> no campo; CSS pinta de vermelho
<code><small id="dica-..."></code> ou <code>regras-...</code>	Dica fixa embaixo do campo	Apenas CSS (texto menor, cinza)
hidden em seções inteiras	Blocos que só aparecem em certos casos	JS mostra quando o caso acontece

Formulários. Os formulários com `novalidate` são validados pelo JS: `evento.preventDefault()`, confere os campos, mostra os erros e só então chama a API. Os de filtro (`role="search"`) também usam `preventDefault()` e atualizam a tabela sem recarregar a página.

Parâmetros de URL. Várias páginas recebem dados pela URL, como `?id=42` ou `?aluno=15`. Leia com `new URLSearchParams(window.location.search).get('id')`.

Segurança (XSS). Todo texto digitado por uma pessoa (descrição do professor, observação do responsável, motivo do admin, nomes) entra na página com `textContent`, nunca com `innerHTML`. Para montar linhas de tabela, crie os elementos com `document.createElement`.

Links. Sempre absolutos, começando com `/` (ex.: `/responsavel/conta.html`), para funcionarem de qualquer pasta.

Valores dos `<select>`. Os `value` de status são iguais ao ENUM do `schema.sql` (rascunho, enviado_revisao, em_revisao, publicado, devolvido, cancelado) e os de bimestre vão de 1 a 4. Mande direto para a API, sem traduzir.

Login e 404

`login.html` é o formulário obrigatório da E1 ("validação antes do envio") e o melhor lugar para começar o JS.

id	Elemento	O JS faz
<code>form-login</code>	<code><form novalidate></code>	Escuta o submit, valida e chama <code>POST /api/login</code>
<code>email</code>	Campo de e-mail	Confere se está preenchido e no formato <code>nome@exemplo.com</code>
<code>senha</code>	Campo de senha	Confere se está preenchido

id	Elemento	O JS faz
erro-email, erro-senha	Erros dos campos	Escreve a mensagem de cada campo
mensagem-geral	Mensagem geral (hidden)	Mostra "E-mail ou senha inválidos" ou erro de conexão

Depois do login, redirecione pelo perfil: administrador → /admin/inicio.html, professor → /professor/inicio.html, responsavel → /responsavel/inicio.html. Durante o envio, desabilite o botão e troque o texto para "Entrando..." (é o "carregamento visível" da E1). O botão ainda não tem id; acrescente id="botao-entrar" no HTML.

404.html não tem JS. O server.js já a entrega com status 404 para qualquer endereço inexistente, e /api/... inexistente responde JSON. Só falta o CSS.

Responsável

Fluxo: **Início** → **Relatórios** (lista com filtros) → **Relatório** (um em detalhe); **Minha conta** pelo menu. É a tela mais usada no celular, então é a prioridade do CSS responsivo.

Página	id	O JS faz
inicio.html	lista-alunos	Um por aluno (nome, turma) com link /responsavel/relatorios.html?aluno=ID
inicio.html	tabela-recentes	Últimos relatórios publicados; 5 colunas; link "Ver relatório" → /responsavel/relatorio.html?id=ID
relatorios.html	form-filtros	Filtros: filtro-aluno, filtro-ano, filtro-bimestre, filtro-disciplina. Ao abrir com ?aluno=, pré-seleciona o aluno
relatorios.html	filtro-aluno, filtro-disciplina	Criar os <option> a partir da API
relatorios.html	resumo-resultados	"12 relatórios encontrados"
relatorios.html	tabela-relatorios	7 colunas; link "Ver relatório" em cada linha

Página	id	O JS faz
relatorios.html	pagina-anterior, pagina-atual, pagina-proxima	Paginação: habilitar/desabilitar botões e escrever "Página 1 de 3"
relatorio.html	mensagem-carregamento	"Carregando..." → apagar; ou erro "Relatório não encontrado ou sem acesso"
relatorio.html	titulo-relatorio	Ex.: "Matemática – 3º bimestre de 2026"
relatorio.html	relatorio-aluno, -turma, -disciplina, -professor, -bimestre, -media, -publicado-em	Um valor em cada <dd>
relatorio.html	relatorio-descricao	Texto do professor, com textContent
relatorio.html	relatorio-tags	Um por tag
relatorio.html	botao-pdf	Baixa o PDF (RF09)
relatorio.html	status-ciencia	"Ciência registrada em ..."; se já registrada, esconder o formulário
relatorio.html	form-ciencia	Validar: checkbox ciencia-confirmacao marcado; ciencia-observacao até 500 caracteres. Erros em erro-confirmacao, erro-observacao; mensagem em mensagem-ciencia
conta.html	conta-nome, conta-email, conta-alunos	Dados da conta e lista de alunos vinculados
conta.html	form-senha	Ver "Formulário de senha" abaixo

Formulário de senha (igual nas três contas): campos senha-atual, senha-nova, senha-confirmacao; erros erro-senha-atual, erro-senha-nova, erro-senha-confirmacao; mensagem mensagem-senha. Regras: todos obrigatórios, nova senha com mínimo de 8 caracteres (12 no admin), com letra e número, e confirmação igual à nova. É o terceiro formulário com validação, se sobrar tempo.

Professor

Fluxo: **Início** → **Minhas turmas** (escolhe turma e vê alunos) → **Acompanhamento** (registra ou edita); **Minha conta** pelo menu. A página `acompanhamento.html` é a mais complexa do projeto.

Página	id	O JS faz
<code>inicio.html</code>	<code>periodo-digitacao</code>	Ex.: "3º bimestre aberto até 30/09/2026 às 23:59"
<code>inicio.html</code>	<code>lista-turmas</code>	Um <code></code> por turma + disciplina, com link <code>/professor/turmas.html?turma=ID&disciplina=ID</code>
<code>inicio.html</code>	<code>form-filtro-status</code> , <code>filtro-status</code>	Filtra a tabela por status
<code>inicio.html</code>	<code>tabela-acompanhamentos</code>	7 colunas; link "Abrir" → <code>/professor/acompanhamento.html?id=ID</code>
<code>turmas.html</code>	<code>form-turma</code> , <code>selecao-turma</code> , <code>selecao-bimestre</code>	Criar os <code><option></code> de turma; ao enviar, buscar os alunos. Pré-selecionar se vier <code>?turma=&disciplina=</code>
<code>turmas.html</code>	<code>resumo-turma</code>	Ex.: "5º A – Matemática · 3º bimestre · 18 de 25 registrados"
<code>turmas.html</code>	<code>tabela-alunos</code>	4 colunas. Sem registro: link "Registrar" com <code>?aluno=&turma=&disciplina=&bimestre=</code> . Com registro: "Abrir" com <code>?id=</code>
<code>acompanhamento.html</code>	<code>titulo-pagina</code>	"Registrar acompanhamento" ou "Editar acompanhamento"
<code>acompanhamento.html</code>	<code>status-acompanhamento</code> , <code>periodo-acompanhamento</code>	Status atual e período; fora do período, bloquear o formulário
<code>acompanhamento.html</code>	<code>bloco-devolucao</code> , <code>motivo-devolucao</code>	Mostrar só se o status for devolvido, com o motivo do admin
<code>acompanhamento.html</code>	<code>form-acompanhamento</code>	Campos <code>campo-turma</code> , <code>campo-aluno</code> , <code>campo-bimestre</code> , <code>campo-descricao</code> , <code>campo-media</code> ; erros <code>erro-* de</code>

Página	id	O JS faz
		cada um; mensagem mensagem-acompanhamento
acompanhamento.html	lista-tags	Criar um checkbox + label por tag (name="tags", value = id da tag)
acompanhamento.html	Botões name="acao"	Os dois são submit: evento.submitter.value diz se foi rascunho ou enviar
acompanhamento.html	secao-retorno, retorno-ciencia, retorno-observacao	Mostrar só se publicado, com a ciência e a observação do responsável
conta.html	conta-nome, conta-email, conta-turmas, form-senha	Igual à conta do responsável, com turmas no lugar de alunos

A página de acompanhamento abre de três jeitos. Sem parâmetros: formulário vazio. Com ?aluno=&turma=&disciplina=&bimestre=: campos pré-preenchidos. Com ?id=: busca o registro e preenche tudo; se o status não for rascunho nem devolvido, o formulário fica só para leitura.

Validação: turma, aluno, bimestre e descrição obrigatórios; descrição até 2000 caracteres; média de 0 a 10 com até uma casa decimal (mesma regra do CHECK no banco).

Administrador

Fluxo: **Início** → **Revisão** (lista com filtros) → **Revisar** (um registro e a decisão); **Minha conta** pelo menu. Cadastros, bimestres e relatórios ficam para a E2.

Página	id	O JS faz
inicio.html	resumo-bimestre, resumo-aguardando, resumo-alunos, resumo-professores	Um número ou texto em cada <dd>
inicio.html	tabela-revisao	7 colunas; link "Revisar" → /admin/revisar.html?id=ID
revisao.html	form-filtros	Filtros filtro-status (já vem em enviado_revisao), filtro-turma, filtro-disciplina, filtro-professor, filtro-bimestre,

Página	id	O JS faz
		filtro-aluno (busca por nome)
revisao.html	filtro-turma, filtro-disciplina, filtro-professor	Criar os <option> a partir da API
revisao.html	resumo-resultados, tabela-acompanhamentos	Resumo e tabela de 8 colunas; link "Revisar" (ou "Ver" se publicado/cancelado)
revisao.html	pagina-anterior, pagina-atual, pagina-proxima	Paginação, igual à do responsável
revisar.html	mensagem-carregamento, titulo-pagina	Carregamento/erro; título como "Ana Souza – Matemática – 3º bimestre"
revisar.html	revisar-aluno, -turma, -disciplina, -professor, -bimestre, -status	Um valor em cada <dd>
revisar.html	form-conteudo	Campos campo-descricao, campo-media, lista-tags (checkboxes já marcados), campo-justificativa; erros erro-*; mensagem mensagem-conteudo
revisar.html	form-decisao	Rádios decisao-publicar, decisao-devolver, decisao-cancelar (name="decisao"); campo-motivo; erros erro-decisao, erro-motivo; mensagem mensagem-decisao
revisar.html	tabela-historico	5 colunas, do mais recente para o mais antigo
conta.html	conta-nome, conta-email, form-senha	Igual às outras contas; senha nova com mínimo de 12 caracteres

Validação de form-conteudo: se a descrição, a média ou as tags mudaram em relação ao que o professor mandou, a justificativa é obrigatória. Guarde os valores originais numa variável quando a página carregar e compare no envio.

Validação de form-decisao: uma opção precisa estar marcada; campo-motivo é obrigatório para devolvido e cancelado. Antes de publicar, vale um `confirm()`, porque a família passa a ver o relatório.

Rotas da API sugeridas

São sugestões para o front e o back combinarem os mesmos endereços; hoje o `server.js` só tem `/teste-db` e a 404. Todas respondem JSON e, em erro, `{ "erro": "mensagem" }`.

Método e rota	Usada em	Devolve
POST <code>/api/login</code>	login	<code>{ usuario: { id, nome, perfil } }</code> ou 401
GET <code>/api/responsavel/alunos</code>	responsável: início, relatórios, conta	Alunos vinculados (id, nome, turma)
GET <code>/api/responsavel/relatorios?aluno=&ano_letivo=&bimestre=&disciplina=&pagina=</code>	responsável: início, relatórios	Lista paginada da view <code>vw_relatorio_publicado</code>
GET <code>/api/responsavel/relatorios/:id</code>	responsável: relatório	Um relatório + tags + ciência
POST <code>/api/responsavel/relatorios/:id/ciencia</code>	responsável: relatório	Registra ciência e observação
GET <code>/api/professor/turmas</code>	professor: início, turmas, acompanhamento, conta	Pares turma + disciplina do professor
GET <code>/api/professor/turmas/:turma/alunos?disciplina=&bimestre=</code>	professor: turmas	Alunos + status do acompanhamento
GET <code>/api/professor/acompanhamentos?status=</code>	professor: início	Acompanhamentos do professor
GET/POST/PUT <code>/api/professor/acompanhamentos/:id</code>	professor: acompanhamento	Ler, criar, editar (acao: rascunho ou enviar)
GET <code>/api/tags</code>	professor e admin	Tags cadastradas
GET <code>/api/admin/resumo</code>	admin: início	Números do resumo

Método e rota	Usada em	Devolve
GET /api/admin/acompanhamentos ?status=&turma=&...&pagina =	admin: início, revisão	Lista paginada
GET/PUT /api/admin/acompanhamentos /:id	admin: revisar	Ler e alterar conteúdo (com justificativa)
POST /api/admin/acompanhamentos /:id/decisao	admin: revisar	Publicar, devolver ou cancelar (com motivo)
GET /api/admin/acompanhamentos /:id/historico	admin: revisar	Linhas da tabela historico
GET /api/conta e PUT /api/conta/senha	as três contas	Dados da conta; troca de senha

Para a E1, basta uma rota real alimentando uma tabela (a E1 aceita API real ou mockada). A mais simples é GET /api/tags ou GET /api/responsavel/relatorios, lendo da view que já existe no banco.

Segurança (E2): o backend precisa conferir o perfil e o vínculo em toda rota. Exemplo: em /api/responsavel/relatorios/:id, confirmar que o relatório é de um aluno daquele responsável e está publicado; senão, 403. A tabela historico ainda não tem coluna para o motivo e a justificativa; avisar quem cuida do banco.

Prioridades até a Entrega 1

A E1 vence em Sep 25, 2026 às 23:59, e cada dia de atraso custa 25% da nota. Faça nesta ordem; os itens marcados com (E1) são exigência direta do documento.

- ☐ **CSS base** (css/style.css, um arquivo para todas as páginas): variáveis de cor do Figma, cabeçalho, menu, formulários, tabelas e mensagens de erro (E1: "CSS organizado")
- ☐ **Responsividade:** mobile first; menu que quebra em linhas no celular; tabelas dentro de um bloco com overflow-x: auto para não estourar a tela (E1: "responsiva")
- ☐ **JS do login:** validação antes do envio e fetch para /api/login (E1: "formulário com validação")
- ☐ **Uma tabela com fetch** mostrando "Carregando..." e erro visível (E1: "consumo assíncrono"); sugestão: tabela-relatorios do responsável
- ☐ **Botão Sair** e #nome-usuario em todas as páginas
- ☐ **Commits de todos os integrantes** no repositório (E1)

Dicas de CSS para esta estrutura:

- Ligue o CSS em todas as páginas com <link rel="stylesheet" href="/css/style.css"> no <head>; hoje só o HTML existe.

- Estilize pelas tags e atributos que já existem (header nav, [aria-current="page"], [aria-invalid="true"], table, dl), assim quase não é preciso mexer no HTML.
- <dl> com <div> dentro vira uma grade de "rótulo: valor" com display: grid.
- [hidden] precisa continuar escondido: se algum display sobrescrever, adicione [hidden] { display: none !important; }.
- Confira o contraste das cores (texto escuro em fundo claro); acessibilidade é requisito do PI.